PRAKTIKUM 3
PERCABANGAN

3.1 Tujuan

1. Mahasiswa mampu menyusun *flowchart* dan program menggunakan struktur percabangan, baik sederhana maupun bersarang, menggunakan berbagai metode percabangan.

3.2 Landasan Teori

3.2.1 Pengantar Percabangan

Struktur percabangan atau seleksi kondisi adalah suatu metode dalam pemrograman yang digunakan untuk mendapatkan keputusan dari salah satu kemungkinan *True* atau *False* (Muhardian, Petani Kode, 2018). Percabangan ini dapat diibaratkan seperti ketika berada pada pertigaan jalan, pengendara harus menentukan harus belok ke kanan atau ke kiri. Hal ini bergantung kepada pengendara ingin berjalan kearah mana. Sama dengan metode percabangan pada pemrograman, bergantung kepada keinginan pemrogram ingin dibawa kemana arah jalannya program.

Contoh:

a) Apabila air sudah mendidih, maka dapat dikatakan air tersebut sudah matang.
b) Apabila suatu nilai lebih dari nol, maka nilai tersebut dinyatakan nilai positif. Sebaliknya, apabila nilai tersebut kurang dari nol, maka nilai tersebut dinyatakan nilai negatif.

Bentuk instruksi percabangan:

a. Instruksi Percabangan Operator
b. Instruksi Percabangan *IF*
    1. Pernyataan *IF* Sederhana
    2. Pernyataan *IF-ELSE*
c. Instruksi Percabangan *CASE*

3.2.2 Instruksi Percabangan Operator

Tidak semua bahasa pemrograman memiliki metode instruksi percabangan menggunakan operator. Bahasa C++ merupakan salah satu bahasa pemrograman yang memiliki metode tersebut.

Tidak hanya pada ilmu pemrograman, pada ilmu matematika pun operator memiliki pengertian umum yang sama. Seperti yang telah dibahas pada bab sebelumnya, operator merupakan sebuah simbol yang digunakan untuk melakukan operasi atau pengolahan atau manipulasi suatu nilai sehingga menghasilkan nilai yang baru.

Percabangan pada bahasa C++ memiliki operator kondisional yang dilambangkan dengan simbol tanda tanya (?) dan simbol titik dua (:) (Muhardian, Petani Kode, 2018). Percabangan operator ini hampir sama dengan percabangan *IF*, namun berbeda tujuannya. Percabangan operator ini digunakan untuk memberikan nilai langsung pada sebuah variabel melalui seleksi kondisi. Percabangan operator sering disebut juga dengan *ternary operator*, yakni operator yang memiliki tiga buah *operand*. Percabangan *IF* akan dijelaskan lebih lanjut pada sub bab berikutnya. Sintaks 3.1 adalah bentuk dasar dari instruksi percabangan operator:

```
nilai1 ? nilai2: nilai3
```

Sintaks 3. 1 *Bentuk Dasar Penulisan Percabangan Operator*

`nilai1` merupakan sebuah kondisi yang menghasilkan keluaran bernilai *boolean* (*True* atau *False*). `nilai2` merupakan pernyataan apabila `nilai1` bernilai *True*. `nilai2` adalah pernyataan apabila `nilai1` bernilai *False*. Pernyataan dalam percabangan operator ini boleh bernilai sembarang. Boleh menggunakan variabel maupun tidak. Untuk lebih jelasnya, perhatikan Sintaks 3.2 berikut:

```
a = (x>=0) ? "bilangan positif": "bilangan negatif"
```

Sintaks 3. 2 *Contoh Sintaks Percabangan Operator*

Pada potongan program pada sintaks 3.2, terlihat bahwa nilai pada variabel `a` akan berubah sesuai dengan hasil seleksi kondisi pada nilai `x`. apabila nilai `x` bernilai lebih dari sama dengan nol, maka variabel `a` akan bernilai "bilangan

positif". Sebaliknya apabila variabel `x` bernilai kurang dari nol, maka variabel `a` akan bernilai "bilangan negatif".

Percabangan operator ini juga dapat digunakan untuk menyeleksi lebih dari satu kondisi. Contohnya seperti pada Sintaks 3.3:

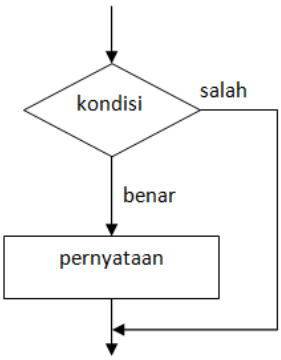
```
a = (x < 0) ? "Kurang dari nol": (x > 0) ? "Lebih
dari nol": "Sama dengan nol"
```

Sintaks 3. 3 *Sintaks Percabangan Operator dengan Lebih Dari Satu Kondisi*

Potongan program pada sintaks 3.3 menggunakan percabangan operator untuk menyeleksi lebih dari satu kondisi. Dimana jika variabel `x` bernilai kurang dari nol, maka nilai variabel `a` akan bernilai "Kurang dari nol", namun jika salah masuk ke seleksi kondisi selanjutnya. Yaitu jika variabel `x` bernilai lebih dari nol, maka nilai variabel `a` akan bernilai "Lebih dari nol". Namun jika salah maka variabel `a` akan bernilai "Sama dengan nol".

3.2.3 Percabangan *IF*

Gambar 3.1 merupakan bentuk *flowchart* percabangan *IF* sederhana secara umum:



Gambar 3. 1 *Potongan Flowchart Percabangan IF*

Potongan *flowchart* pada Gambar 3.1 dapat diartikan bahwa apabila kondisi bernilai benar, maka program akan menjalankan blok pernyataan. Namun apabila kondisi bernilai salah, maka program akan melanjutkan ke instruksi selanjutnya tanpa menjalankan blok pernyataan.

Sintaks 3.4 merupakan penulisan sintaks percabangan *IF*:

```
if(kondisi) {
      pernyataan;
      …
}
```

Sintaks 3. 4 *Sintaks Percabangan IF*

Potongan program pada sintaks 3.4 memiliki pengertian atau jalannya program yang sama dengan *flowchart* diatas sebelumnya. Yakni pernyataan yang berada diantara blok "{" dan "}" hanya akan dikerjakan apabila kondisi bernilai *True* atau benar.

Perlu diketahui, pada praktiknya, penulisan "{" dan "}" pada blok pernyataan tidak selalu dibutuhkan. Penggunaan "{" dan "}" hanya diperlukan apabila blok pernyataan berisi lebih dari 1 baris.

Contoh 1:

```
#include <iostream>

using namespace std;

int main() {
     int usia;
     cout << "Berapa usia Anda: ";
     cin >> usia;

     if (usia < 17)
         cout << "Anda tidak boleh menonton bioskop";
}
```

Sintaks 3. 5 *Contoh Program Percabangan IF sederhana*

Tampilan program ketika dijalankan:

```
Berapa usia Anda: 15
Anda tidak boleh menonton bioskop
Process returned 0 (0x0)   execution time : 0.857 s
Press any key to continue.
```

Gambar 3. 2 *Tampilan Program Ketika Sintaks 3.5 Dijalankan*

Contoh 2:

```
#include <iostream>

using namespace std;

int main() {
     int usia;
     cout << "Berapa usia Anda: ";
     cin >> usia;
     if (usia < 17) {
          cout << "Anda tidak boleh menonton bioskop"<<endl;
          cout << "Anda belum cukup umur";
     }
}
```

Sintaks 3. 6 *Contoh Lain Program Percabangan IF sederhana*

Tampilan program ketika dijalankan:

```
Berapa usia Anda: 16
Anda tidak boleh menonton bioskop
Anda belum cukup umur
Process returned 0 (0x0)   execution time : 3.159 s
Press any key to continue.
```

Gambar 3. 3 *Tampilan Program Ketika Sintaks 3.6 Dijalankan*

Pada *statement IF* terdapat pula kondisi *ELSE IF* yang bisa digunakan untuk menyeleksi kondisi lain selain *IF*. Serta *ELSE* yang digunakan apabila semua kondisi tidak terpenuhi (*False*). Untuk lebih jelas, perhatikan contoh pada Sintaks 3.7:

```
if (kondisi1){
      perintah1;
      ...
}
else if (kondisi2) {
      perintah2;
      ...
}
else{
      perintah3;
      ...
}
```

Sintaks 3. 7 *Sintaks Penulisan Percabangan IF Majemuk*

Dari contoh pernyataan *IF* pada Sintaks 3.7, dapat diartikan sebagai berikut:

a) Jika `kondisi1` bernilai benar, maka program akan menjalankan `perintah1`.

b) Jika `kondisi1` bernilai salah, maka program hanya akan melewati blok `perintah1` (tidak dikerjakan) dan bergerak ke baris selanjutnya, yaitu melihat hasil keluaran seleksi kondisi untuk `kondisi2`.

c) Apabila `kondisi2` bernilai benar, maka program akan menjalankan `perintah2`.

d) Apabila `kondisi1` dan `kondisi2` bernilai salah, maka secara otomatis program akan menjalankan `perintah3` tanpa menjalankan `perintah1` dan `perintah2`.

Penulisan pada Sintaks 3.7 bisa juga disebut dengan bentuk pernyataan *IF* majemuk. Sintaks 3.8 merupakan contoh lain program yang didalamnya menggunakan pernyataan *IF* majemuk:

Contoh 3:

```
#include <iostream>

using namespace std;

int main() {
     int usia;
     cout << "Berapa usia Anda: ";
     cin >> usia;
     if (usia < 17) {
          cout << "Anda tidak boleh menonton bioskop" <<
          endl;
          cout << " Anda belum cukup umur ";
     } else {
          cout << "Anda boleh menonton bioskop."<<endl;
          cout << "Selamat menonton";
     }
}
```

Sintaks 3. 8 *Contoh Lain Percabangan IF Majemuk*

Tampilan program ketika dijalankan:

```
Berapa usia Anda: 15
Anda tidak boleh menonton bioskop
 Anda belum cukup umur
Process returned 0 (0x0)   execution time : 1.500 s
Press any key to continue.
```

Gambar 3. 4 *Tampilan Program ketika Sintaks 3.8 Dijalankan jika Keadaan IF Terpenuhi*

```
Berapa usia Anda: 17
Anda boleh menonton bioskop.
Selamat menonton
Process returned 0 (0x0)   execution time : 3.380 s
Press any key to continue.
```

Gambar 3. 5 *Tampilan Program ketika Sintaks 3.8 Dijalankan jika Keadaan ELSE Terpenuhi*

Selain format penulisan *statement IF* seperti pada Sintaks 3.8, ada pula bentuk pernyataan *IF* bersarang (*Nested IF*), seperti pada Sintaks 3.9:

```
if (kondisi1){
      if (kondisi1a){
            perintah1a;
            ...
      }
      else {
            perintah1b;
            ...
      }
}
else {
      if (kondisi2){
            perintah2a;
            ...
      }
      else {
            perintah2b;
            ...
      }
}
```

Sintaks 3. 9 *Sintaks Percabangan IF Bersarang*

Berikut penjelasan dari contoh pada Sintaks 3.9:

1. Apabila `kondisi1` bernilai benar, maka program akan menjalankan seleksi kondisi lagi untuk `kondisi1a`.

2. Apabila `kondisi1a` bernilai benar, maka program akan menjalankan `perintah1a`. sebaliknya, jika kondisi1a bernilai salah, maka program akan menjalankan `perintah1b`.
3. Apabila `kondisi1` bernilai salah, maka program akan menjalankan seleksi kondisi untuk `kondisi2`.
4. Apabila `kondisi2` bernilai benar, maka program akan menjalankan `perintah2a`. Sebaliknya, apabila `kondisi2` bernilai salah, maka program akan menjalankan `perintah2b`.

3.2.4 Percabangan *Switch-Case*

Pernyataan *SWITCH-CASE* adalah salah satu jenis pernyataan yang dapat digunakan untuk melakukan seleksi kondisi. Secara struktur, *SWITCH-CASE* memiliki struktur yang lebih ringkas dibanding dengan pernyataan *IF*. Untuk lebih jelasnya perhatikan contoh perbandingan bentuk *IF* dan *SWITCH-CASE* pada Sintaks 3.10:

```
if (ekspresi == konstanta_1) {
      pernyataan_1;
      ...
} else if (ekspresi == konstanta_2) {
      pernyataan_2;
      ...
} else {
      pernyataan_lain;
      ...
}
```

```
switch (ekspresi) {
     case konstanta_1:
       pernyataan_1;
       break;
     case konstanta_2:
       pernyataan_2;
       break;
     default:
       pernyataan_lain;
}
```

Sintaks 3. 10 *Perbandingan Penulisan Percabangan IF dan Percabangan SWITCH-CASE*

Pada percabangan *SWITCH-CASE*, *SWITCH* bertugas sebagai penampung nilai yang akan diseleksi. Sedangkan *CASE* bertugas sebagai penyeleksi atau pembanding antara nilai yang ada dalam *SWITCH* dan nilai-nilai baru yang akan menjadi acuan, pada Sintaks 3.10 `konstanta_1` dan `konstanta_2`. Cara kerjanya sama, apabila nilai `ekspresi` sama dengan `konstanta_1` maka program akan menjalankan `pernyataan_1`. Setelah itu program akan keluar dari percabangan *SWITCH-CASE* karena ada pernyataan `break`. Namun apabila nilai `ekspresi` sama dengan `konstanta_2` maka program akan menjalankan `pernyataan_2`. Setelah itu program akan keluar dari percabangan *SWITCH-CASE* karena ada pernyataan `break`. Apabila nilai `ekspresi` tidak sama dengan `konstanta_1` maupun `konstanta_2`, maka `default` akan bernilai benar dan mengerjakan `pernyataan_lain`.

Dari penjelasan tersebut, dapat disimpulkan bahwa fungsi penggunaan `break` pada *SWITCH-CASE* sama dengan penggunaan *`else if`* pada *IF*, yakni apabila salah satu kondisi sudah terpenuhi (bernilai benar), maka program akan keluar dari percabangan. Sedangkan fungsi penggunaan `default` pada *SWITCH-CASE* sama dengan penggunaan *`else`* pada *IF*, yakni untuk memberikan kondisi apabila tidak ada kondisi yang terpenuhi (bernilai benar).

Beberapa hal yang perlu diperhatikan pada penggunaan *SWITCH-CASE* adalah:

a) Pada *keyword* *`case`* harus diisi dengan sebuah konstanta, tidak bisa diisi dengan pernyataan kondisional (seperti *range*).

b) Konstanta yang digunakan harus bertipe data *int* atau *char* (tidak bisa menggunakan *string* atau *double*/*float*).

c) Apabila *keyword* `break` digunakan, maka setelah program mengeksekusi salah satu pernyataan (salah satu kondisi bernilai benar), maka program akan langsung keluar dari pernyataan *SWITCH-CASE*.

d) Pernyataan `default` akan bernilai benar apabila `konstanta_1` dan `konstanta_2` bernilai salah.

Sintaks 3.11 merupakan contoh penggunaan *SWICTH-CASE* dalam program:

```
#include <iostream>

using namespace std;

int main () {
      int n;

      cout << "Masukkan bilangan: ";
      cin >> n;

      switch (n) {
            case 1:
                  cout << "Anda memasukkan angka satu";
                  break;
            case 2:
                  cout << "Anda memasukkan angka dua";
                  break;
            case 3:
                  cout << "Anda memasukkan angka tiga";
                  break;
            default:
                  cout << "Anda memasukkan angka selain
                  1, 2, dan 3";
                  break;
      }
}
```

Sintaks 3. 11 *Penggunaan Percabangan SWITCH-CASE dalam Program*

Tampilan program ketika dijalankan:

```
Masukkan bilangan: 1
Anda memasukkan angka satu
Process returned 0 (0x0)   execution time : 1.092 s
Press any key to continue.
```

Gambar 3. 6 *Tampilan Program Ketika Sintaks 3.11 Dijalankan Ketika Kondisi Case Terpenuhi*

```
Masukkan bilangan: 10
Anda memasukkan angka selain 1, 2, dan 3
Process returned 0 (0x0)   execution time : 0.931 s
Press any key to continue.
```

Gambar 3. 7 *Tampilan Program Ketika Sintaks 3.11 Dijalankan Ketika Kondisi Case Tidak Terpenuhi*

Berikut merupakan bebepara perbedaan yang paling jelas antara percabangan *IF* dan percabangan *SWITCH-CASE*:

a) Percabangan *IF* dapat mengerjakan seleksi kondisi yang berupa operasi logika atau kondisional, sedangkan percabangan *SWITCH-CASE* tidak.

b) Struktur percabangan *SWITCH-CASE* lebih efektif dan lebih ringkas penulisannya apabila digunakan untuk menyeleksi lebih dari 2 kondisi sederhana.

c) Percabangan *SWITCH-CASE* dan percabangan *IF* dapat dikombinasikan untuk dimasukkan kedalam satu bagian, dengan catatan tidak ada kesalahan dalam penulisan sintaks atau notasi.

d) Percabangan *SWITCH-CASE* tidak dapat melakukan seleksi terhadap kondisi yang bertipe data *string* / kalimat maupun bernilai *double* atau bilangan pecahan.

3.3 Tugas Praktikum

Tugas praktikum sengaja dikosongkan.

PRAKTIKUM 4
PERULANGAN

4.1 Tujuan

1. Mahasiswa mampu menyusun *flowchart* dan program menggunakan struktur perulangan, baik sederhana maupun bersarang, menggunakan berbagai metode perulangan.

4.2 Landasan Teori

4.2.1 Pengantar Perulangan

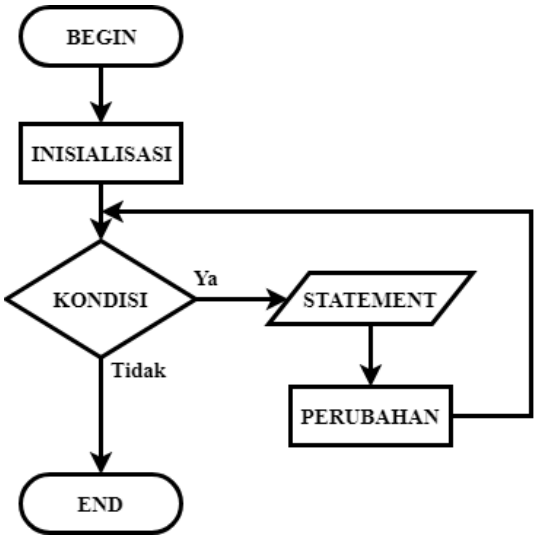
Perulangan (*looping*) atau iterasi adalah sebuah proses dalam program untuk menjalankan sebuah blok instruksi secara berulang-ulang atau berkali-kali sebanyak batas yang telah ditentukan (Universitas Dian Nuswantoro, n.d.). Hal ini dapat digunakan untuk meringkas penulisan program karena instruksi hanya dituliskan 1x tetapi program akan menjalankan berkali-kali sesuai batas yang ditentukan. Misalkan ingin menampilkan tulisan yang sama sebanyak 100x, maka tidak perlu menuliskan sintaks untuk menampilkan tulisan tersebut sampai 100 baris. Cukup dengan menuliskan sintaks untuk mencetak tulisan tersebut 1x, lalu dimasukkan ke dalam blok perulangan yang diulang sebanyak 100x. Hal ini tentu akan memudahkan serta meringkas penulisan kode program.

Terdapat dua jenis perulangan berdasarkan jumlah perulangannya. Yang pertama perulangan yang telah diketahui batas perulangannya (secara matematis). Jenis perulangan ini menggunakan perulangan *FOR*. Sedangkan jenis perulangan yang kedua adalah perulangan yang belum diketahui batas perulangannya. Jenis perulangan ini menggunakan perulangan *WHILE* dan *DO WHILE*. Secara konsep, *WHILE* dan *DO WHILE* hampir sama. Perbedaannya terletak pada kondisi perulangan yang diperiksa di awal dan di akhir perulangan. Pada perulangan *WHILE*, kondisi perulangan diperiksa di awal perulangan, sama seperti *FOR*. Sedangkan pada perulangan *DO WHILE*, kondisi perulangan diperiksa di akhir perulangan. Sehingga apabila menggunakan *DO WHILE*, pernyataan akan dikerjakan setidaknya sebanyak 1 kali.

4.2.2 Pernyataan *FOR*

Pernyataan *FOR* digunakan pada perulangan yang telah diketahui batas perulangannya secara matematis (MateriDosen, 2017). Misalnya diulang sebanyak 5 kali, atau diulang selama nilai tidak lebih besar daripada b, dan lain sebagainya. Selain batas perulangan, yang perlu diketahui dari awal sebelum melakukan perulangan adalah nilai atau kondisi awal variabel yang akan dijadikan acuan untuk batas perulangan, serta perubahan pada variabel acuan tersebut apabila kondisi perulangan terpenuhi dan *statement* telah dikerjakan.

Bentuk umum:



Gambar 4. 1 *Flowchart Perulangan FOR*

```
for(inisialisasi;kondisi;perubahan){
      statement;
}
```

Sintaks 4. 1 *Format Penulisan Sintaks Perulangan FOR*

Berikut merupakan penjelasan sekilas mengenai bagian-bagian dari proses perulangan *FOR*:

a) inisialisasi: digunakan untuk memberikan nilai atau kondisi awal sebuah variabel yang akan dijadikan acuan untuk menentukan batas perulangan.

b) kondisi: digunakan sebagai pengendali perulangan, kondisi akan menentukan apakah perulangan akan dilanjutkan (*statement* dikerjakan) atau dihentikan.

c) perubahan: digunakan sebagai pengubah nilai variabel pengendali perulangan apabila *statement* telah dijalankan. Perubahan nilai bisa berupa kenaikan atau penurunan nilai pencacah.

d) *statement*: sebuah blok program yang dijalankan ketika kondisi perulangan terpenuhi.

Contoh program:

```
#include <iostream>

using namespace std;

int main()  {
    int angka;

    cout << "Angka     Angka kuadrat \n";
    cout << "------------------------------\n";

    for (angka = 1; angka <= 10; angka++)
        cout << angka << "\t\t" << (angka * angka) << end
        l;

    return 0;
}
```

Sintaks 4. 2 *Sintaks Perulangan FOR Sederhana*

Tampilan program ketika dijalankan:

```
Angka     Angka kuadrat
_____________________
1              1
2              4
3              9
4              16
5              25
6              36
7              49
8              64
9              81
10             100

Process returned 0 (0x0)   execution time : 0.298 s
Press any key to continue.
```

Gambar 4. 2 *Tampilan program ketika Sintaks 4.2 Dijalankan*

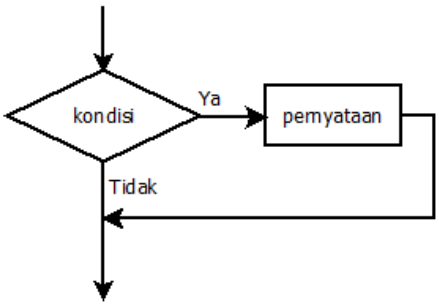
4.2.3 Pernyataan *WHILE*

Jika dilihat berdasarkan urutan pengerjaan program, struktur perulangan *WHILE* memiliki urutan pengerjaan yang sama dengan struktur perulangan *FOR*. Perbedaannya terletak pada sudah diketahui atau belum batas perulangannya. Apabila sudah diketahui batas perulangannya, sebaiknya menggunakan *FOR*. Sebaliknya apabila belum diketahui batas perulangannya, maka dapat menggunakan *WHILE* (Santoso, 2017). Pada perulangan *WHILE*, syarat atau kondisi diperiksa terlebih dahulu, apabila terpenuhi, maka *statement* dijalankan. Setelah itu program akan mengulang dari awal, kembali ke periksa kondisi. Begitu seterusnya hingga kondisi tidak terpenuhi, maka perulangan dihentikan. Disini yang perlu jadi catatan, pemrogram harus teliti dan hati-hati. Karena tidak seperti pada perulangan *FOR*, dimana perubahan variabel pengendali diberikan blok khusus, sehingga pemrogram akan selalu ingat untuk menuliskannya. Pada perulangan *WHILE*, perubahan nilai untuk variabel pengendali berada di dalam blok *statement*. Sehingga penting sebagai pengingat agar tidak lupa untuk menambahkan blok perubahan nilai untuk variabel pengendali perulangan di dalam blok *statement* agar tidak menjadi *infinity loop* ketika program dijalankan.

Bentuk umum perulangan *WHILE* dapat dilihat pada Sintaks 4.3:

```
while (syarat/kondisi){
    pernyataan1;
    pernyataan2;
    ...
    perubahan;
}
```

Sintaks 4. 3 *Format Penulisan Sintaks Perulangan WHILE*



Gambar 4. 3 *Flowchart Perulangan WHILE*

Contoh program:

```
#include <iostream>

using namespace std;

int main()  {
    int n = 1;

    while(n <= 10)
    cout << n++ << " ";
    return 0;
}
```

Sintaks 4. 4 *Sintaks Perulangan WHILE Sederhana*

Tampilan program ketika dijalankan:

```
1 2 3 4 5 6 7 8 9 10
Process returned 0 (0x0)   execution time : 0.280 s
Press any key to continue.
```

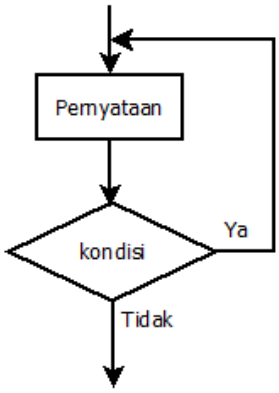
Gambar 4. 4 *Tampilan program ketika Sintaks 4.4 Dijalankan*

4.2.4 Pernyataan *DO-WHILE*

Bentuk umum perulangan *DO-WHILE* dapat dilihat pada Sintaks 4.5 dan Gambar 4.3:

```
do {
    pernyataan1;
    pernyataan2;
    ...
    perubahan;
} while (syarat/kondisi);
```

Sintaks 4. 5 *Format Penulisan Sintaks Perulangan DO WHILE*



Gambar 4. 5 *Flowchart Perulangan DO WHILE*

Sama dengan perulangan *WHILE*, perulangan *DO WHILE* digunakan untuk perulangan yang belum diketahui batas perulangannya. Bedanya, pada perulangan *WHILE*, kondisi diperiksa terlebih dahulu sebelum menjalankan blok *statement* (Santoso, 2017). Apabila kondisi bernilai salah, maka *statement* tidak dijalankan atau perulangan berhenti. Sedangkan dalam perulangan *DO WHILE*, *statement* dijalankan terlebih dahulu, kemudian program akan menjalankan blok perubahan nilai variabel pengendali, selanjutnya barulah program memeriksa nilai kondisi. Hal ini mengakibatkan blok *statement* akan dijalankan setidaknya 1 kali.

Sama dengan perulangan *WHILE*, penting sebagai pengingat agar pemrogram tidak lupa untuk menambahkan blok perubahan nilai untuk variabel pengendali perulangan di dalam blok *statement* agar tidak menjadi *infinity loop* ketika program dijalankan.

Contoh Program:

```
#include <iostream>

using namespace std;

int main(){
    int n = 1;

    do {
        cout << n++ << " ";
    } while(n <= 10);
    return 0;
}
```

Sintaks 4. 6 *Sintaks Perulangan DO WHILE Sederhana*

Tampilan program ketika dijalankan:

```
1 2 3 4 5 6 7 8 9 10
Process returned 0 (0x0)   execution time : 0.275 s
Press any key to continue.
```

Gambar 4. 6 *Tampilan program ketika Sintaks 4.6 Dijalankan*

4.2.5 Pernyataan *BREAK*

Pernyataan *BREAK* digunakan untuk berhenti atau keluar secara paksa dari suatu perulangan tidak peduli apakah kondisi masih terpenuhi atau tidak.

Pernyataan *BREAK* bisa digunakan untuk semua jenis perulangan, baik perulangan *FOR*, *WHILE*, maupun *DO WHILE*.

Apabila perintah *BREAK* dijalankan, maka program akan menjalankan *statement* berikutnya yang terletak setelah blok perulangan. Untuk lebih jelasnya perhatikan Sintaks 4.7 berikut:

```
int n = 0;

while (n++ < 10) {
     cout << n << endl;

     if (n == 5)
          break;
}
```

Sintaks 4. 7 *Contoh Format Penulisan Sintaks BREAK*

Program pada Sintaks 4.8 adalah contoh penggunaan perintah *Break* dalam pernyataan perulangan *for*. Bilangan deret dapat dihentikan menggunakan perintah *Break*.

```
#include <iostream>

using namespace std;

int main()  {
     int n= 0;
     int angka;

     for (angka=0; angka < 20; angka++)   {
          n += angka;

          if (n >= 100)
               break;
     }

     cout << "Deret Bilangan: 1 + 2 + ... + " << angka
     << endl;
     cout << "Jumlah Deret Bilangan = " <<n;

     return 0;
}
```

Sintaks 4. 8 *Contoh Program Penggunaan Sintaks BREAK*

Tampilan program ketika dijalankan:

```
Deret Bilangan: 1 + 2 + ... + 14
Jumlah Deret Bilangan = 105
Process returned 0 (0x0)   execution time : 0.264 s
Press any key to continue.
```

Gambar 4. 7 *Tampilan program ketika Sintaks 4.8 Dijalankan*

4.3 Tugas Praktikum

Tugas praktikum sengaja dikosongkan.